```sql
-- car_sales_queries.sql
-- Purpose: clean Bright Motors raw CSV, create processed table and key aggregates

-- Adjust syntax slightly depending on your SQL engine (Snowflake / Postgres / MySQL)

-- 1) Create a staging table (example for Snowflake / Postgres)
CREATE TABLE IF NOT EXISTS staging_bright_car_sales (
   sale_id VARCHAR,
   sale_date DATE,
   year INTEGER,
   make VARCHAR,
   model VARCHAR,
   trim VARCHAR,
   body_type VARCHAR,
   transmission VARCHAR,
   vin VARCHAR,
   state VARCHAR,
   condition_rating NUMERIC,
   odometer VARCHAR,
   exterior_color VARCHAR,
   interior_color VARCHAR,
   seller VARCHAR,
   mmr_value VARCHAR,
   selling_price VARCHAR,
   units_sold INTEGER DEFAULT 1
);

-- 2) Load CSV into staging (engine-specific; use your bulk loader)
```

```
-- For Snowflake: use COPY INTO staging_bright_car_sales FROM
@%staging_bright_car_sales FILE_FORMAT=(TYPE=CSV...)


-- 3) Clean and normalize numeric columns, remove commas/currency symbols
CREATE TABLE IF NOT EXISTS bright_car_sales_clean AS
SELECT
    sale_id,
    sale_date,
    year,
    make,
    model,
    trim,
    body_type,
    transmission,
    vin,
    state,
    TRY_CAST(NULLIF(condition_rating, '') AS NUMERIC) AS condition_rating,
    NULLIF(odometer, '') AS odometer_raw,
    CASE
        WHEN odometer ~ '^[0-9,]+' THEN CAST(REPLACE(odometer, ',', '') AS
INTEGER)
        ELSE NULL
    END AS odometer_km,
    exterior_color,
    interior_color,
    seller,
    CASE WHEN mmr_value IS NULL OR mmr_value = '' THEN NULL
         ELSE CAST(REPLACE(REPLACE(mmr_value, '$', ''), ',', '') AS NUMERIC)
    END AS mmr_value,
    CASE WHEN selling_price IS NULL OR selling_price = '' THEN NULL
```

```
        ELSE CAST(REPLACE(REPLACE(selling_price, '$', ''), ',', '') AS NUMERIC)
    END AS selling_price,
    COALESCE(units_sold, 1) AS units_sold
FROM staging_bright_car_sales;

-- 4) Create processed table with revenue and profit columns
CREATE TABLE IF NOT EXISTS bright_car_sales_processed AS
SELECT
    *,
    selling_price * units_sold AS total_revenue,
    -- Placeholder cost price calculation: assume cost = 0.8 * selling_price when cost not provided.
    COALESCE(NULLIF(NULL, NULL), selling_price * 0.8) AS cost_price,
    ROUND( (selling_price - (selling_price * 0.8)) / NULLIF(selling_price,0) * 100, 2) AS profit_margin_pct,
    CASE
        WHEN ROUND( (selling_price - (selling_price * 0.8)) / NULLIF(selling_price,0) * 100, 2) >= 20 THEN 'High Margin'
        WHEN ROUND( (selling_price - (selling_price * 0.8)) / NULLIF(selling_price,0) * 100, 2) BETWEEN 10 AND 19.99 THEN 'Medium Margin'
        ELSE 'Low Margin'
    END AS margin_tier,
    DATE_TRUNC('month', sale_date) AS sale_month,
    DATE_TRUNC('quarter', sale_date) AS sale_quarter
FROM bright_car_sales_clean;

-- 5) Example aggregate: revenue by make and model
CREATE OR REPLACE VIEW v_revenue_by_make_model AS
SELECT
    make,
```

```sql
    model,
    SUM(total_revenue) AS total_revenue,
    SUM(units_sold) AS units_sold,
    AVG(selling_price) AS avg_price
FROM bright_car_sales_processed
GROUP BY make, model
ORDER BY total_revenue DESC;

-- 6) Example aggregate: price vs mileage correlation sample (per engine support)
-- Many SQL engines do not offer correlation; Snowflake supports CORR()
SELECT CORR(selling_price, odometer_km) AS price_mileage_corr FROM bright_car_sales_processed WHERE selling_price IS NOT NULL AND odometer_km IS NOT NULL;

-- 7) Regional sales
CREATE OR REPLACE VIEW v_regional_sales AS
SELECT state AS region, COUNT(*) AS transactions, SUM(total_revenue) AS revenue
FROM bright_car_sales_processed
GROUP BY state
ORDER BY revenue DESC;

-- 8) Export/Download: instruct your SQL engine to export bright_car_sales_processed to Excel/CSV for visualization
-- Snowflake: use COPY INTO @my_stage/bright_car_sales_processed.csv FROM bright_car_sales_processed FILE_FORMAT=(TYPE=CSV,HEADER=TRUE);

-- End of script
```